

Curso

Desarrollo Web en HTML5, CSS3 y JavaScript (avanzado)

Módulo 1:

HTML5 avanzado

Unidad 1:

Repaso sobre los conceptos básicos de HTML5 y CSS3



Presentación

Con la última versión de HTML (HTML5) se incorporaron nuevas herramientas a las existentes que nos facilitan el trabajo, evitando utilizar programas de terceros y disponiendo ahora de soporte nativo.

Una de estas mejoras que trajo esta nueva versión de HTML se encuentra en los elementos de vídeo y audio. En esta unidad abordaremos dichas mejoras para que podamos utilizar todos los recursos que nos provee HTML5 en estos elementos.



Objetivos

Que los participantes logren...

- Incorporen y repasen los conceptos de HTML5, CSS3 y JavaScript.
- Identifiquen los conceptos de tal forma que puedan diferenciar entre HTML5, CSS3 y JavaScript.



Bloques temáticos

1.HTML5: estructura y elementos .

- a. Etiquetas:definición y tipo
- b. Etiquetas semánticas ,estructurales y funcionales
- c. Atributos

2.CSS3 : dándole estilo al sitio.

- a. Selectores
- b. Colores
- c. Unidades
- d. Animación en CSS
- e. Responsive Design

3.JavaScript : creando comportamiento al sitio.

- a. Etiqueta <noscript>
- b. Variables y funciones
- c. Operadores
- d. Condicionales
- e. Array y Objeto
- f. Estructuras de repetición
- g. Funciones Nativa

1. HTML5: estructura y elementos

El HTML (Lenguaje de Marcas de Hipertexto, del inglés *HyperText Markup Language*) es el componente básico de un sitio web. Este lenguaje nos permite definir el contenido y la estructura de un sitio web. ¿A qué nos referimos con esta definición? Cuando nosotros vemos un sitio, podemos detectar que tiene ciertos elementos que lo forman y que se acomodan en un cierto orden (por ejemplo: primero vemos un logo, un menú, luego un título, después una imagen, etc). Este orden y estos elementos se definen con el HTML.

El HTML es entonces el tronco inicial y fundamental del sitio, donde vamos a ver que podemos incluso vincular otros archivos y, con ayuda de otros lenguajes, darle estilos y funcionalidad a esos elementos que lo componen.

Tengamos en cuenta que HTML fue creado alrededor de 1989, por lo que a medida del paso del tiempo fue mejorando su lenguaje y en la actualidad contamos con la versión HTML5 que es la que utilizamos en este curso.

En caso de querer saber más sobre la historia de HTML, dejamos el siguiente link donde lo explican de manera sencilla y rápida:

<https://www.youtube.com/watch?v=B5VjL2f1IQg>

El HTML se conforma de etiquetas y atributos, los cuales definiremos en el siguiente punto.

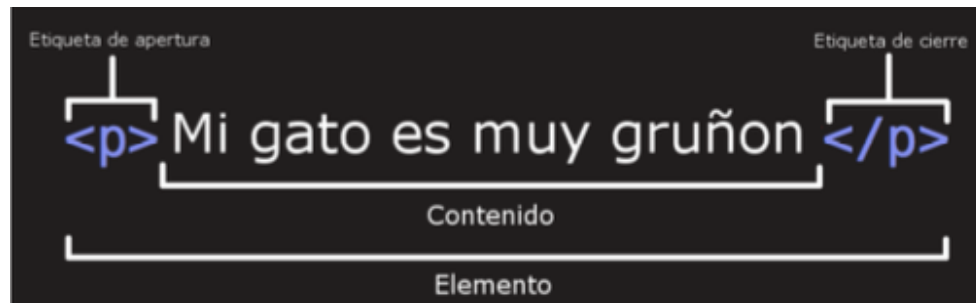
Etiquetas: definición y tipo

En el punto anterior habíamos definido el HTML como un lenguaje que definía, entre otras cosas, los elementos del sitio. Para que el navegador pueda identificar los distintos elementos del sitio, HTML nos provee de una marcación que cataloga los elementos.

Estas marcas se llaman **etiquetas**, contienen el nombre del elemento en minúscula rodeado por los símbolos “<” y “>”.

Las etiquetas suelen encerrar al elemento aportando información necesaria al navegador sobre el mismo. Contamos con dos formas de etiquetar que se utilizan dependiendo de qué elemento es.

Por un lado, tenemos aquellas que tienen una etiqueta de apertura y otra de cierre, encerrando entre ambas etiquetas al contenido del elemento, como vemos en la imagen de ejemplo.



Por otro lado, tenemos aquellas que tienen una única etiqueta que funciona de apertura y de cierre. En vez de encerrar el contenido, utiliza otras herramientas para definirlo, como vemos en la imagen de ejemplo:

apertura y cierre de etiqueta

`<img src= "images/firefox-icon.png" alt= "Mi imagen de prueba" />`

A continuación, dejamos una breve lista de algunas de las etiquetas de HTML más utilizadas. También aclaramos si se escriben como una única etiqueta o si poseen etiqueta de cierre.

	ETIQUETA	DESCRIPCION
Etiquetas que poseen etiqueta de cierre	<code><h1>, <h2>, <h3>, <h4>, <h5>, <h6></code>	Titulo o encabezados - el numero indica la jerarquia donde <code><h1></code> es el mas grande e importante y <code><h6></code> el menos importante. IMPORTANTE por cada HTML solo puede haber un solo <code><h1></code>
	<code><p></p></code>	Utilizada para indicar bloques de texto o parrafos
	<code></code>	Define una lista de items desorganizada
	<code></code>	Define una lista de items organizada, suelen ser numeradas pero puede organizarse de manera alfabetica
	<code></code>	Etiqueta utilizada para indicar los items de las listas
	<code><a></code>	Ancla - Sirve para redireccionar a otra pagina ya sea secciones de la misma pagina u otra pagina - email o telefonos
	<code><button></button></code>	Boton - Etiqueta que define un boton
etiquetas sin etiqueta de cierre	<code></code>	Sirve para introducir imagenes dentro del documento
	<code><hr/></code>	Dibuja una linea horizontal. Se utiliza cuando se quiere separar contenido
	<code>
</code>	Utilizado para generar un salto de linea
	<code><input/></code>	Etiqueta utilizada para crear controles interactivos en formularios

Etiquetas semánticas, estructurales y funcionales

Ahora que ya recordamos lo que es una etiqueta y cómo la escribimos, podemos clasificarlas en tres grupos, según la información y funcionalidad que cumplen : **semánticas**, **estructurales** y **funcionales**.

Por un lado, las etiquetas **semánticas** son aquellas que tienen un significado en el mundo real, las que indican al navegador dónde estamos parados y qué es exactamente lo que queremos mostrar en el contenido.

Algunas de ellas son : <p> ; <h1><h2>...; <header>;<nav>; <button>, entre otras.

Dentro de este grupo de etiquetas se encuentran algunas que nos permitirán darle un sentido de orden y maquetar nuestro HTML.

Es decir que, al igual que una noticia tiene sectores como encabezado, cuerpo, pie de página, etc., el HTML tiene la posibilidad de seccionar el documento en sectores que le brindan al sitio una estructura más robusta, una aplicación de buenas prácticas y una mejor accesibilidad en nuestro sitio.

Recordemos que accesibilidad se refiere a que todo tipo de usuario pueda acceder a nuestro sitio y tenga una buena experiencia. Algunos de nuestros usuarios pueden utilizar programas como screen readers, que necesitan de este tipo de etiquetas para contextualizar al usuario del sector en donde se encuentra, haciendo más amena la navegación del usuario por el sitio.

Las etiquetas utilizadas en la maquetación son: <header>; <nav>;<main>;<section>;<aside>;<article> y <footer>.

Por su parte, las etiquetas **estructurales** son aquellas etiquetas que funcionan como contenedores de otras, son como “cajas” que nos permiten agrupar varias etiquetas y nos sirven para organizar el contenido e incluso para dar estilos a bloques. Ellas son : <div> y .

Por último, las etiquetas **funcionales** son aquellas que nos permiten agregar funcionalidad al elemento. Algunas de ellas son : <a> ; .

1.3 Atributos

Los **atributos** son palabras reservadas o valores que agregan información acerca del elemento e incluso pueden utilizarse para definir el contenido de las etiquetas que no poseen etiqueta de cierre.

Teniendo en cuenta esto, debemos indicar que alguna de ellas serán obligatorias para que el elemento exista (como por ejemplo en la etiqueta , definir la ruta en donde se encuentra dicha imagen), mientras que otras le aportaran información complementaria al elemento.

Estos atributos se definen usualmente con un nombre en minúscula y muchas veces necesitan tener asignado un valor; para ello, debemos escribir el atributo el signo igual y luego el valor entre comillas.

Contamos con una amplia variedad de atributos. Dejamos a continuación algunos de ellos.

Atributo ▼	Etiqueta en la que se suele usar ▼	Utilidad ▼
alt	<u><area>, , <input></u>	Atributo que da una breve descripción de la imagen
class	<u>Atributo Global, en todas las etiquetas</u>	Utilizado para darle nombre a una etiqueta. Este atributo nos permite nombrar a varias etiquetas con el mismo nombre
href	<u><a>, <area>, <base>, <link></u>	Atributo que especifica la url a la cual linkea esa pagina.
id	<u>Atributo Global, en todas las etiquetas</u>	Utilizado para darle un nombre único e irrepetible a una etiqueta.
src	<u><audio>, <embed>, <iframe>, , <input>, <script>, <source>, <track>, <video></u>	Atributo que nos va a indicar la ruta a hacer para encontrar el archivo que queremos

2. CSS3: dándole estilo al sitio

El CSS (Hojas de estilo en cascada o Cascading Style Sheets) es el lenguaje que se encarga de darle estilo a nuestro sitio. Con él vamos a poder modelarlo y darle distintas propiedades para que cambien su apariencia.

Como estamos hablando de nuestro sitio, no podemos dejar afuera al HTML, el lenguaje que utilizamos para crear la estructura y el contenido de nuestro sitio.

El HTML nos va a permitir incluir CSS en él para poder estirar nuestros elementos.

Para incluirlo contamos con dos formas:

- Incluir la etiqueta `<style>` que contiene todo el lenguaje CSS correspondiente a ese HTML (forma menos recomendable).
- Incluir la etiqueta `<link>` en el `<head>` que nos permitirá vincular un archivo externo CSS donde se encuentren los estilos del sitio (forma más recomendable ya que hace que el código quede más prolijo y amigable a la vista).

¿Cómo escribimos CSS?

Para escribir CSS, vamos a utilizar la siguiente estructura: un **selector** (lo definiremos en el siguiente punto), unas **llaves** (`{}`) y las **propiedades** que va a recibir ese elemento. Estas últimas se escriben indicando el nombre de la propiedad y su valor separado por dos puntos (`:`)



2.1 Selectores

Para poder identificar a los elementos del HTML y poder darles propiedades, CSS nos brinda lo que llamamos **selectores**.

Un selector es un patrón de elementos que indican al navegador qué elementos HTML se seleccionan para aplicarles alguna propiedad. Con CSS tenemos la posibilidad de darle estilos tanto a una etiqueta en particular como a un grupo de ellas. Para ello, nos provee de cuatro tipos de selectores:

- **SELECTORES DE ETIQUETAS**

Son los selectores que indican el nombre de la etiqueta directamente. Estos selectores hacen que las propiedades que coloquemos dentro de las llaves afecten a todo elemento que tenga dicha etiqueta.

- **SELECTORES DE CLASES**

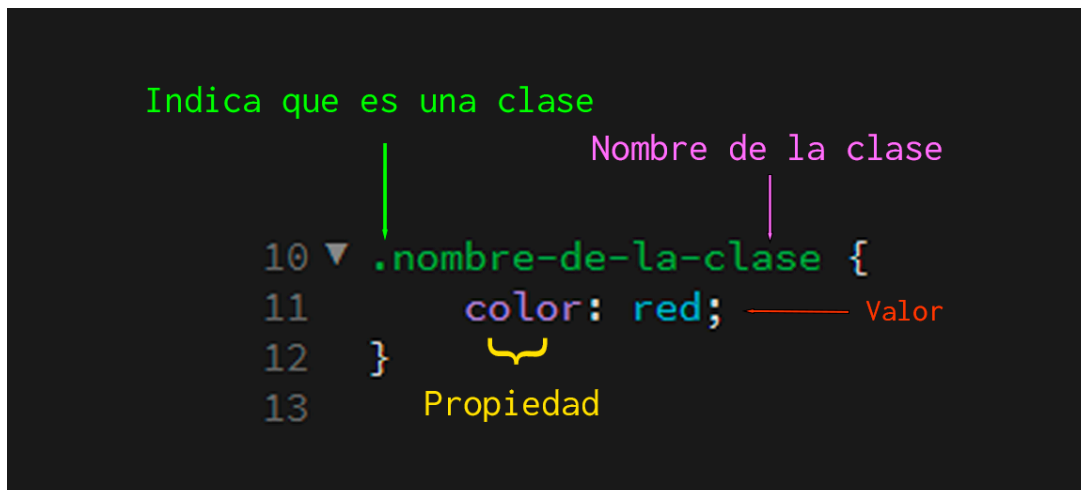
Como vimos en el cuadro de atributos, las clases nos permiten llamar a varias etiquetas con el mismo nombre. La cualidad de este atributo es que podemos darle el mismo nombre a varias etiquetas y

así “agruparlas”.

En CSS, esto nos va a servir para que podamos darle las mismas propiedades a todos los elementos que compartan dicha clase.

Para indicarle a CSS qué ese selector corresponde a una clase, debemos colocar un punto (.) delante del nombre de esa clase.

Sintaxis:



- **SELECTORES DE ID**

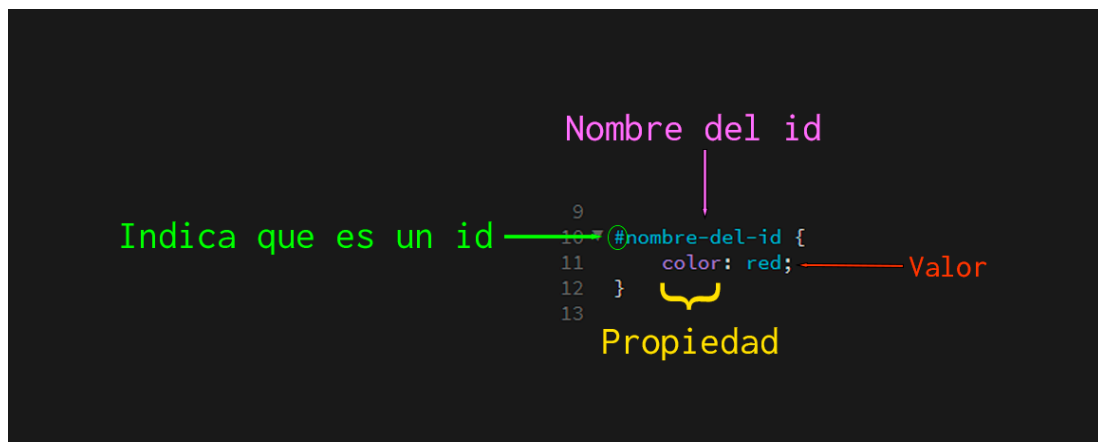
Cuando hablamos de ID, nos referimos a un atributo que nos permite nombrar de forma única a un elemento, diferenciándolo del resto.

Este atributo se puede comparar con el número de DNI de una persona, debe ser único e identifica a una persona en particular. En el caso de los ID, tienen un nombre único en todo el documento y diferencian a un elemento o etiqueta en particular.

Con este tipo de selectores, podemos definir las propiedades que solamente tendrá el elemento o etiqueta que posea dicho ID.

Para indicarle a CSS que ese selector corresponde a una clase, debemos colocar el símbolo numeral (#) delante del nombre de ese ID.

Sintaxis:



● SELECTORES COMBINADOS

Este tipo de llamado a elementos o etiquetas se caracteriza por el uso de símbolos que nos permiten agrupar selectores o especificar los mismos. Tenemos varios de ellos pero a continuación nombraremos a los más utilizados. En caso de querer saber más sobre este tipo de [selectores, ingresen aquí](#).

Por un lado, contamos con la posibilidad de indicar que a varios selectores se les asignen las mismas propiedades. Para ello,

necesitamos listar a todos los selectores a estilar, separados por una coma (,).

Sintaxis:

```
1
2
3 nombreDelSelector , otroSelector{
4
5     color: red;
6
7 }
```

Por otra parte, contamos con la posibilidad de indicar que solamente un elemento que se encuentra dentro de otro específico contenga esas propiedades (esto se denomina el hijo de un elemento padre). Para ello, necesitamos indicar el selector que hace referencia al elemento padre, el signo mayor que (>) y el selector hijo.

Sintaxis:

```
nombreDelSelectorPadre > otroSelectorHijo{
    color: red;
}
```

También tenemos la posibilidad de indicarle que queremos que un elemento que se encuentre al lado de otro (es decir, el elemento hermano) contenga propiedades. Para ello, colocamos el elemento

de referencia y el elemento a asignarle las propiedades, separados por el signo más (+) .

Sintaxis:

```
1  
2  
3 nombreDelSelectorHermano + otroSelectorHermano{  
4  
5     color: red;  
6  
7 }  
8
```

Teniendo en cuenta que con estos datos ya podemos asignar propiedades a los elementos, dejamos a continuación una lista de algunas de las propiedades de CSS.

PROPIEDAD	DESCRIPCION
padding	Propiedad que abrevia todas las propiedades de estilo del padding.El padding es el relleno del elemento, donde se encuentra lo que colocas en la etiqueta.
border	Propiedad que abrevia todas las propiedades de estilo del borde del elemento
margin	Propiedad que abrevia todas las propiedades de estilo del margin del elemento.El margin es el espacio que se encuentra por fuera del border
font-family	Propiedad que indica la familia tipografica del selector en el que se define - que tipo de letra va a tener.
font-size	Propiedad que indica el tamaño de la tipografía.
font-weight:	Propiedad que indica el peso de la tipografía.
text-align	Propiedad que indica la ubicación de la tipografía
width	Especifica el ancho de los elementos
height	Especifica la altura del elemento
display	Propiedad que permite indicar el comportamiento del selector en linea o en bloque.Tambien permite ocultar el elemento.
background-color	Propiedad que indica el color de fondo del elemento
background-image	Indica la imagen que va a estar de fondo del selectoren el que se define
background	Propiedad que abrevia todas las propiedades de estilo de fondo a la vez.Puede indicar desde color,imagen,repeticion,etc.
border-radius	Propiedad que permite redondear los bordes del elemento
color	Define el color de la tipografia del selector en el que se lo define.
box-sizing	Propiedad que permite ajustar el borde y relleno del elemento
z-index	Propiedad que permite manipular el plano de los elementos.Cual se encuentra por encima del otro.

Para más información sobre estas propiedades u otras, pueden acceder al [siguiente link](#).

2.2 Colores

En CSS tenemos varias formas de poder definir colores, pero aquí definiremos las más utilizadas:

- **RGB /RGBA:**

Sigla para **Red-Green-Blue** (colores primarios luminosos). Se toman tres parámetros, uno para cada color, y unidos muestran un color final.

Cuando usamos el **RGBA**, lo que estamos haciendo es agregarle un cuarto parámetro que va a ser la **transparencia** que queramos agregar.

Por ejemplo:

color: rgba(168, 56, 159, 10%);
color: rgb(66, 209, 94);

- **Coding hexadecimal:**

Es un formato que utiliza 6 dígitos luego de un numeral. Cada dos dígitos, se representa una graduación de cada uno de los tres colores primarios luminosos para dar como resultado final un color en particular.

Esos dígitos van del 0 a F y se pueden definir utilizando distintas combinaciones.

Por ejemplo:

El color blanco sería #FFFFFF.

El color negro es #000000.

Existen páginas que ayudan a encontrar el valor en formato hexadecimal del color que queremos, ya que es muy poco común saberlos de memoria. Algunas de ellas son:

<https://flatuicolors.com/>

<https://color.adobe.com/es/create/color-wheel#>

<https://www.cdmon.com/es/tabla-colores>

- **Nombres según HTML**

HTML nos brinda 140 colores ya definidos con ciertos nombres.

Por ejemplo:

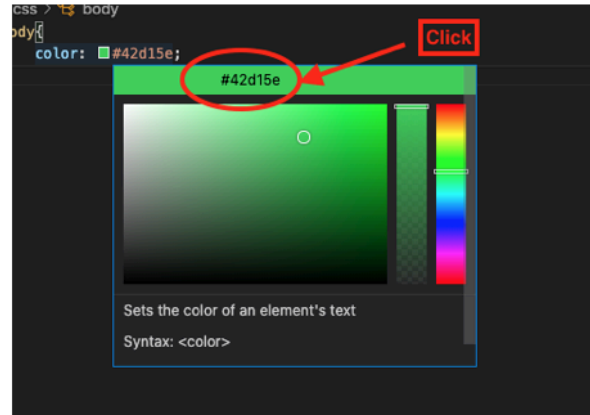
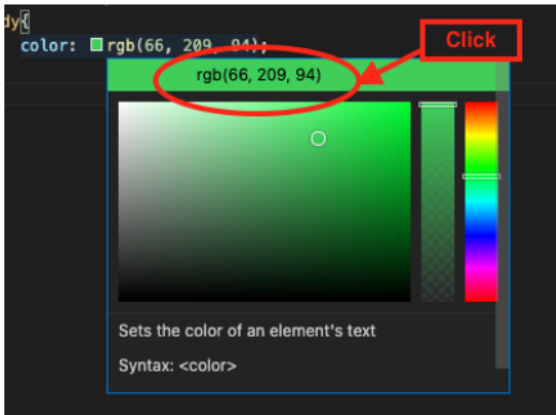
color: red; (se muestra el color rojo)

color: aquamarine; (se muestra un color turquesa)

Importante: CSS permite modificar el formato del color; es decir que si tenemos un color definido en rgb podemos cambiarlo a hexadecimal.

Si coincide con uno de los colores definidos por HTML, podemos ver cuál es su valor en rgb o hexadecimal.

Lo haremos parando sobre el color, y en la pantalla que aparece, clickeando en su nombre.



2.3 Unidades

En el siguiente apartado vamos a indicar algunas de las unidades más utilizadas para indicar medidas en CSS.

Ellas son:

- **PIXELES (px)**

Es el bloque de construcción más pequeño de una pantalla. Esta unidad se caracteriza por ser un valor numérico fijo y escribirse seguido de la sigla **px**.

- **PORCENTAJE(%)**

Es una unidad relativa que va tomar como 100% el tamaño del elemento padre en donde se encuentre contenido nuestro selector. A partir de ello, calculará el tamaño que va a tener nuestro elemento según el porcentaje indicado.

- **VIEWPORT HEIGHT Y VIEWPORT WIDTH (vh y vw)**

Viewport height y Viewport width son unidades relativas que, a diferencia del porcentaje, van a tomar como 100% el **alto (height)** o el **ancho (width)** de la pantalla del navegador, en vez del tamaño del contenedor padre.

Para ver más tipos de unidades pueden visitar este sitio:
<https://www.eniun.com/unidades-medida-css/>

2.4 Animación en CSS

Una de las características más interesantes que nos brinda CSS3 es la posibilidad de crear nuestras propias animaciones a los elementos del sitio sin la necesidad de programas ni plugins.

Para ello contamos con las siguientes propiedades. En caso de necesitarlo, podemos incluso combinarlas dentro del mismo elemento.

ANIMATION

Esta propiedad nos permite crear la animación para elementos de HTML estableciendo distintos estadios de esta durante el tiempo que deseemos que dure.

Para ello utiliza la palabra clave **@keyframe** que asigna tanto el nombre de la animación como la lista de estilos que va a tener a lo largo del tiempo y en qué momento de la animación va a tenerlos. Esto último lo podemos determinar utilizando **porcentajes** donde indicamos el estadio inicial, estadios intermedios y el final; o bien indicando con las palabras claves **from** y **to** para indicar solamente el estadio inicial y final.

Sintaxis:

```
@keyframe nombre_animacion{  
    0%{  
        propiedad_del_elemento_en_dicha_instancia  
    }  
    25%{  
        propiedad_del_elemento_en_dicha_instancia  
    }  
    50%{  
        propiedad_del_elemento_en_dicha_instancia  
    }  
    100%{  
        propiedad_del_elemento_en_dicha_instancia  
    }  
}
```

[Video explicativo del tema.](#)

Una de las cualidades más notorias de @keyframe es que, al definir la animación por fuera de cualquier selector, podemos utilizarlo en los elementos que necesitemos sin tener que definirlos, simplemente invocándolos.

Teniendo en cuenta cómo podemos definir un `@keyframe`, estamos en posición de poder listar de una manera resumida las propiedades de animation que nos permite desde invocar la animación definida en los `@keyframe` hasta agregar propiedades puntuales sobre la animación en los elementos.

PROPIEDAD ANIMATION	DESCRIPCION
<code>animation-name</code>	Propiedad que invoca a la animacion creada con el <code>@keyframe</code>
<code>animation-duration</code>	Propiedad que indica el tiempo que debe durar la animación
<code>animation-delay</code>	Propiedad que define el tiempo de demora que tiene la animacion antes de comenzar.
<code>animation-iteration-count</code>	Indica las veces que debe repetirse la animacion
<code>animation</code>	Propiedad abreviada de todas las propiedades de animation.

TRANSFORM

Esta propiedad de CSS nos permite modificar o transformar un elemento en términos bidimensionales y tridimensionales. Nos permite trasladar, rotar, escalar o incluso inclinar elementos.

A continuación listamos de una manera resumida las propiedades de transform:

TRANSITION

Esta propiedad nos sirve para indicar cómo queremos que se produzca el cambio entre dos estados de un elemento.

Para definirlo, debemos indicar la propiedad que produce el cambio y la duración. Este último utiliza como unidad los segundos (**s**) y tenemos que tener en cuenta que si no definimos la duración este tomará su valor

por defecto que es 0 segundos (**0s**), lo que provoca que no se produzca el efecto.

METODOS TRANSFORM	DESCRIPCION
<code>translate(tX, tY):</code>	traslada el elemento en 2D. Puede tomar valores de longitud.
<code>rotate(deg)</code>	metodo nos permite girar el elemento.
<code>scale(sX, sY) / scale(sXY)</code>	nos permite escalar en dos dimensiones al elemento
<code>scaleX(sX) / scaleY(sY)</code>	metodo que nos permite escalar el elemento en un eje ("x" o "y")
<code>skew(sX,sY):</code>	método nos permite inclinar al elemento en 2D

Sintaxis

transition: *nombre_de_la_propiedad duración*;

2.5 Responsive Design

Llamamos Responsive Design a las técnicas que nos permiten hacer webs adaptables a las condiciones de cada dispositivo que accede a ellas. Esto se realiza mediante el uso de **media queries** una vez que se haya definido el estilo para un tamaño de pantalla inicial.

Cuando hablamos de **media queries**, nos referimos a declaraciones que se definen en CSS y funcionan como un condicional en el estilo, indicando que propiedades deben modificarse al verse el sitio en un cierto tipo de dispositivos.

Estas declaraciones constan de la palabra reservada **@media** junto con el punto en donde el diseño va a cambiar entre paréntesis, seguido de unas llaves que contienen las propiedades a modificar.

Sintaxis

```
@media breakpoint {  
  
    propiedades_que_cambian  
}
```

Este punto llamado **breakpoint** es el ancho de pantalla en el que las “nuevas” propiedades van a aplicarse.

Recordemos que estas técnicas están enfocadas al **ancho de pantalla** de cada dispositivos.

Para definir breakpoints contamos con tres posibilidades:

- *tipo_de_dispositivo** and (min-width: *ancho_minimo_de_pantalla*)
- *tipo_de_dispositivo** and (max-width: *ancho_maximo_de_pantalla*)

Puede tomar como valores:

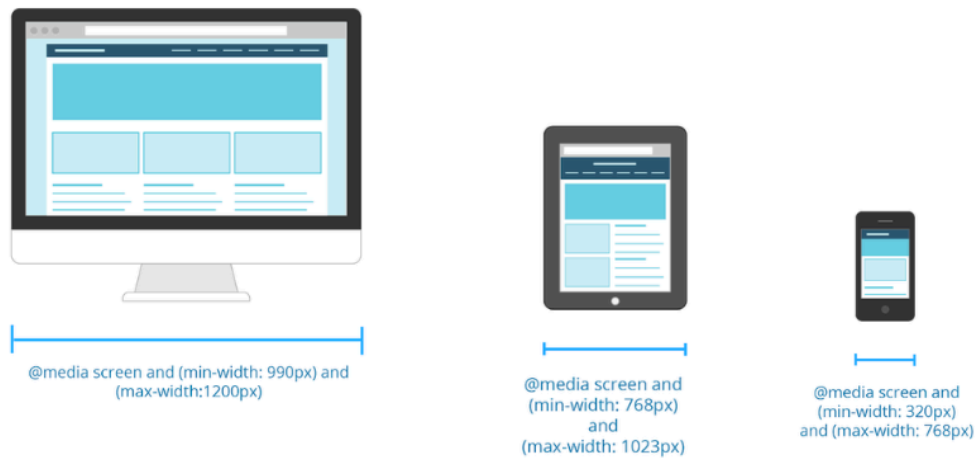
- **screen** (que hace referencia a pantallas de computadora, tablets y smartphones),
 - **print** (que hace referencia a impresiones).
 - **speech** (que hace referencia a screen readers).
 - **all** (que se usa para todo tipo de dispositivos de media)
-
- (max-width: *ancho_maximo_de_pantalla*) ó (min-width: *ancho_minimo_de_pantalla*)

Como pudimos ver en las posibilidades de definir breakpoints, contamos con dos opciones para indicar el ancho de pantalla: **min-width** (qué sería: “cuando el tamaño de la pantalla tenga como mínimo este tamaño”) y **max-width** (qué sería: “cuando el tamaño de la pantalla tenga como máximo este tamaño”).

Para saber cuándo es recomendable utilizar uno y cuándo utilizar el otro, les proponemos definir si el estilo del sitio va a estar pensado primero para una computadora y luego adaptarla a dispositivos más pequeños; o si se encuentra pensado para verlo inicialmente en celulares y luego adaptar el tamaño para dispositivos más grandes (Mobile First).

De esta forma, en caso de que el sitio inicie con un estilo pensado para computadora, recomendamos utilizar el max-width ; mientras que si el sitio inicia con el estilo pensado para mobile, recomendamos utilizar el min-width.

En cuanto al tamaño de ancho de pantalla, les dejamos una orientación del ancho estándar de alguno de los dispositivos para que puedan tener como guía al momento de pensar y hacer los breakpoints:



ACLARACIÓN:

Se pueden agregar los breakpoints que se deseen, es decir que podemos tener desde dos @media hasta incluso una cantidad infinita de breakpoints para tamaños de pantalla intermedios.

3. JavaScript: Creando comportamiento en el sitio

JavaScript es un lenguaje multiplataforma que nos permite agregar funcionalidad a nuestro sitio. Este lenguaje se caracteriza por ser poco tipado (es decir que no tiene muchas reglas a diferencia de otros lenguajes, lo que nos permite tomarnos ciertas libertades al escribir en este lenguaje), permitirnos interactuar con el usuario (es decir, del lado del cliente) y con el servidor (es decir, del lado del backend); y es muy utilizado en el ámbito laboral.

Para el propósito de estos cursos, nos enfocamos en la parte que interactúa con el usuario por medio de la pantalla del navegador y nos va a permitir realizar acciones tales como: recibir información de formularios, que el usuario pueda cambiar el aspecto del sitio a voluntad, que el sitio reaccione cuando el usuario haga alguna acción sobre un elemento, que realice una cuenta matemática, etcétera

Para integrar JavaScript contamos con tres opciones:

- Por medio de una etiqueta `<script>` al final del `<body>`, donde utilizando el atributo `src` vincule un archivo javascript externo (opción que recomendamos ya que evita lentitud al momento de la carga del sitio, se aplican buenas prácticas, hace que el código sea más prolijo y amigable al momento de visualizarlo).
- Por medio de la misma etiqueta `<script>` al final del `<body>`, donde entre la etiqueta de apertura y de cierre contenga el código JavaScript.
- Por medio de una etiqueta `<link>` dentro del `<head>` donde se vincule a un archivo externo escrito en JavaScript, que tendrá

la extensión **.js** , hace referencia al tipo de lenguaje en el que se encuentra escrito.

3.1 Etiqueta <noscript>

Aunque la mayoría de los navegadores hoy en día son capaces de leer JavaScript; existen algunos como el Explorer que no disponen del soporte completo de JavaScript o tiene bloqueado su uso.

Por tal motivo se suele recomendar agregar la etiqueta <noscript> en el cuerpo del HTML (preferentemente al inicio del body), que engloba un mensaje que escribimos de aviso al usuario, indicando que debería activar JavaScript o cambiar de navegador para poder tener una mejor experiencia en la página, activándose en caso que el navegador no pueda ejecutar JavaScript.

3.2 Variables y funciones

Variables

Cuando hablamos de variables, nos referimos a espacios de memoria que nos sirven para guardar un valor en la computadora.

Estos espacios de memoria tienen asignado un nombre (que puede ser el que más nos parezca apropiado- se recomienda un nombre que haga referencia al valor que contenga), y con ello asociado la información o valor a guardar.

Para hacerlo más visual, podemos pensar en las variables como cajas que las etiquetamos, donde se guarda información y podemos usar más adelante.

Sintaxis

keyword nombreDeVariable = valor;

Como podemos ver en la sintaxis, aparece un valor *keyword*. Este valor corresponde a una palabra reservada que hace referencia a que lo definido luego corresponde a una variable.

Esta *keyword* puede tomar tres valores, dependiendo del tipo de uso que le vayamos a dar al dato durante el documento:

- **var:** Palabra clave con la que se indican variables en los inicios de JavaScript. Esta variable nos permite poder acceder al dato desde cualquier función del archivo JavaScript, modificarlo e incluso definir una variable nueva con el mismo nombre.

Al ser tan permisivo, se recomienda evitar su uso, ya que puede provocar errores; por ejemplo, al querer armar una nueva variable sin acordarnos que ya ese nombre se encontraba ocupado.

- **let:** Palabra clave con la que se indican variables que pueden cambiar su contenido durante el transcurso del archivo JavaScript.

Esta variable nos permite, en una primera instancia, poder reservar un espacio de memoria con una información y luego ir modificando su contenido a lo largo del archivo de JavaScript. Una de las cualidades de este tipo de variables es que tiene una limitación de su alcance (se denomina **scope**) permitiendo acceder y modificarlo solamente teniendo en cuenta donde se encuentre definido,.

Esto quiere decir que, si definimos esta variable dentro de una función, la misma solamente existe mientras se ejecute la función y solo podemos acceder a su contenido dentro de esta, desapareciendo una vez que finalizó su ejecución. En cambio, si

la misma se define por fuera de una función, existe a lo largo de todo el archivo JavaScript y puede ser accedido a su contenido desde cualquier función o acción en JavaScript.

- **const:** Palabra clave con la que se indica que la variable va a tener ese mismo dato guardado a lo largo de todo el archivo JavaScript. Este tipo de variables se utiliza cuando el valor que contienen no se modificará, solamente pudiendo utilizarse ese dato definido.

En cuanto a su alcance (**scope**), cuenta con la misma característica que **let**, pudiendo acceder a su valor dentro de la función o del archivo en que se encuentre, dependiendo de donde se esté definido. Recordemos que en este caso solamente vamos a poder acceder al valor y no modificarlo.

Las funciones son bloques de código que nos permiten realizar ciertas tareas. Son un **conjunto de instrucciones** que pueden tomar valores de entrada y, según necesitemos, pueden devolver un resultado.

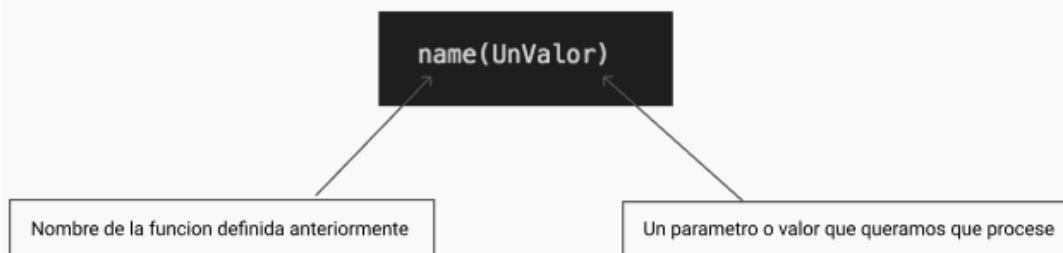
Para poder utilizarlas, debemos primero definir las en algún lugar de nuestro archivo de JavaScript y luego invocarlas las veces que necesitemos. De esta manera, las funciones nos permiten reutilizar código y optimizar tiempos de redacción, entre otras cosas.

3.3 Operadores

Los operadores son aquellos que nos permiten obtener una información a partir de dos valores, es decir que a partir de un valor vamos a obtener un tercero.

Contamos con una amplia variedad de operadores, tienen asociados distintos símbolos según el caso. En esta sección vamos a ocuparnos de los tres grupos principales de operadores:

Como se invocan ??



Como se definen ??

Palabra reservada
"function"

nombre que queramos darle

valor/es de entrada

```
function name(params){  
    //desarrollo de la funcion  
    return // palabra reservada para devolver lo que queramos  
};
```

valor/es de entrada

nombre que queramos darle

```
// Otra forma de escribirlo  
var name = (params) => {  
    //desarrollo de la funcion  
    return // palabra reservada para devolver lo que queramos  
};
```

Aritmeticos →

Son aquellos que ya conocemos que nos permiten usar numeros y realizar operaciones con ellos.

Utiliza los simbolos :

- + (suma)
- (resta)
- * (multiplicacion)
- / (division)

De Comparacion

Son aquellos que producen una respuesta afirmativa(true) o negativa(false) a una pregunta de comparacion
Utilizada para comparar dos valores

Ellos son:

Igual a (==) Solo compara el valor
Igual a (===) Compara valor y tipo de valor.
Distinto a (!=) Solo compara valor
Distinto a (!==) Compara valor y tipo.
Mayor a (>) // Menor a (<)
Mayor o igual a (>=) // Menor o igual a (<=)

Logicos

Son aquellos que toman operaciones y la evaluan,dando como resultado una respuesta afirmativa (true),o negativa (false).

Ellos son:

- **AND (&&):** Nos permite preguntarnos si dos cosas son **simultaneamente true**.
- **OR (||):** Nos permite preguntarnos **si al menos alguna** de las dos cosas son **true**.
- **NOT (!):** Nos permite preguntarnos si algo **no es lo que esperabamos que fuera**.

3.4 Condicionales

En el código se pueden presentar situaciones en donde queremos que el programa tome caminos dependiendo de ciertas circunstancias, por lo que necesitamos que el programa sea capaz de “**tomar decisiones**”. Para ello JavaScript nos ofrece lo que llamamos **condicionales**, una estructura que evalúa una condición que definimos para que realice o no una acción.

Contamos con tres tipos de estructuras de condicionales:

- **Operadores ternarios**

Operador que se compone por una condición y las acciones a realizar si la condición resultaba verdadera (**true**) o falsa (**false**).

Este tipo de operador permite únicamente realizar una acción en cada caso y necesita sí o sí que le indiquemos qué acción realizar tanto para un resultado verdadero como un resultado falso, ya que sino no va a poder ejecutarse correctamente.

SINTAXIS

*condición ? accionEnCasoDeSerLaRespuestaTrue :
accionEnCasoDeSerLaRespuestaFalse*

- **if - if/else**

En el operador **if**, este se compone por una condición y las acciones a realizar si la condición resultaba verdadera (**true**). A diferencia del operador ternario, no es necesario indicar qué acción se realizará en caso de que la condición sea falsa. Nos permite realizar más de una acción.

En cuanto a la estructura ***if/else***, se refiere a un “agregado” que se le puede hacer a la estructura ***if*** para poder manipular las acciones que queremos que el programa realice en caso que el resultado de la condición de falso, cubriendo ese faltante que no puede realizar el operador ternario.

SINTAXIS

```
if ( condición ) {  
    Ejecuto un código  
}
```

Si la condición es **true**, ejecuto el código entre llaves.

```
if ( condición ) {  
    Ejecuto si se cumple la condición  
} else {  
    Ejecuto si no se cumple la condición  
}
```

- **switch**

Este operador, a diferencia de los anteriores, no evalúa que una condición sea verdadera o falsa, sino que evalúa una expresión y posee distintas opciones de ejecución según la respuesta o valor obtenido de esa evaluación.

Esta estructura permite en cada uno de los casos posibles de respuesta, ejecutar múltiples acciones y solo permite evaluar valores concretos.

SINTAXIS

```
switch (key) {  
    case value:  
        break;  
    case otherValue:  
        break;  
    default:  
        break;  
}
```

Donde:

- La cláusula **default** es opcional pero recomendable, ya que presenta las instrucciones que se ejecutarán en caso que la respuesta a la expresión no se encuentre definida en los demás casos.
- **break** permite que se corte la ejecución y no siga evaluando si los demás casos cumplen con la respuesta de la expresión.
- **Switch** tiene en cuenta el tipo de dato y el valor de dato en cada caso. También permite expresiones como booleanos.

3.5 Array y Objeto

Array

Los Array son estructuras que nos permiten almacenar una gran cantidad de información en forma de una lista, donde mediante distintos métodos vamos a poder tratar a cada uno de los valores que lo componen.

De esta manera, vamos a poder agrupar datos que se relacionen entre sí, haciendo más simple su uso.

Nos permite almacenar cualquier tipo de dato y no tiene límite en la cantidad de datos a almacenar.

En cuanto a su sintaxis, se compone por corchetes donde se encuentran listados los datos separados por una coma (exceptuando el último valor) y se guardan en variables.

SINTAXIS

```
const  
  o nombre_variable =[  
cualquier_tipo_de_dato_1,cualquier_tipo_de_dato_2,  
...,cualquier_tipo_de_dato_n]  
let
```

Para poder manipular o acceder a los valores de estos arrays, JavaScript nos provee de una amplia variedad de métodos. Vamos a repasar los más utilizados:

Length() : Función que nos permite ver la cantidad de valores que tiene un array.

Ej: listaSuper.length() → 4

Push() : Función que nos permite agregar un valor **al final** del array.

Ej: listaSuper.push("harina") → ["pan", "queso", true, Cheetos, "harina"]

Pop() : Función que nos permite **sacar el valor final** del array.

Ej: listaSuper.pop() → ["pan", "queso", true]

Splice(índice, cantidad) : Función que nos permite **sacar varios valores a la vez**.

*Ej: listaSuper.slice(1,2)→ ["pan", "queso", true, Cheetos]
["pan", Cheetos]*

Unshift() : Función que nos permite **agregar un valor al inicio del array**

*Ej: listaSuper.unshift("harina") → [**"harina"**, "queso", "pan", true, Cheetos]*

Shift() : Función que nos permite **sacar el valor inicio del array**

Ej: listaSuper.shift() → ["queso", true, Cheetos]

IndexOf() : Función que nos permite **buscar en qué posición está un valor del array**

Ej: listaSuper.indexOf(Cheetos) → 3

0	1	2	3
listaSuper= ["pan", "queso", true, Cheetos]			

Objetos

Los objetos son estructuras que guardamos en variables que nos permiten almacenar múltiples propiedades de distintos datos.

Los objetos se escriben entre llaves ({}), donde dentro coloca el nombre de la propiedad (denominada **claves** o **keys**), seguido por dos puntos (:) y luego el valor de esa propiedad (denominado **value** o **valor**). Cada **clave** o **key** con su **valor** correspondiente se separa con una coma (,) a excepción de la última propiedad.

SINTAXIS

```
var/let/const  nombreDelObjeto ={  
    nombreDeLaClave : suValor ;  
    nombreDeOtraClave : suValor  
}
```

Al momento de acceder a estas propiedades, JavaScript nos ofrece dos formas de hacerlo:

- **nombreDelObjeto.nombreDeLaClave :**

Donde colocamos el nombre del objeto un punto y luego el nombre de la propiedad

- **nombreDelObjeto[nombreDeLaClave] :**

Donde colocamos el nombre del objeto y luego, entre corchetes, el nombre de la propiedad

3.6 Estructura de repetición

Al momento de trabajar con arrays necesitamos poder manejar estructuras que nos permitan manipularlos de forma masiva para evitar un trabajo extenso y excesivo. Para ello, JavaScript nos provee de estructuras llamadas **bucles**, que son una porción de código que se ejecuta

cíclicamente a partir de una condición. Esta condición va a permitir no solamente que se inicie la ejecución sino que finalice, haciendo que el ciclo tenga un principio y un final, evitando una repetición infinita.

Estas estructuras son el **for**, **for/of**, el **while** y el **do/while**.

- **for**

Este bucle está compuesto por una condición que se coloca dentro de paréntesis, y unas llaves que contienen lo que se ejecutará en caso de cumplirse dicha condición.

La condición consiste de 3 partes:

- Una expresión inicial, donde se establece que se realizará antes de iniciar el bucle (usualmente se suele iniciar una variable para armar un contador).
- La condición que va a definir en cada vuelta si se entra al bucle o no.
- Una expresión final, que se ejecuta al finalizar cada ciclo del bucle.

```
for (expresion_inicial; condicion ; expresion_final) {  
    /*  
        Acciones a realizar cuando se cumple la condicion.  
        Accion ciclica.  
    */  
}
```

- **for/of**

Este bucle es similar al **for** pero con la diferencia de que se encarga de recorrer la totalidad de la lista que indiquemos. No tiene una

condición que pueda indicarle que recorra una porción del array.

Esta estructura está compuesta por una condición que se coloca entre paréntesis y posee un nombre aleatorio que utilizaremos para referirnos a cada ítem del array, la palabra **of** y el nombre del array a recorrer; luego unas llaves que contienen lo que se ejecutará con cada ítem de dicho array.

```
for (nombreReferenciaAlItem of nombreDelArray) {  
    /*  
        Acciones a realizar en cada elemento que compone el array.  
        Accion ciclica.  
    */  
}
```

- **while**

Este bucle está compuesto por una condición que se coloca dentro de paréntesis y unas llaves que contienen lo que se ejecutará en caso de cumplirse dicha condición, al igual que la estructura **for**. La única diferencia será que solamente contiene la condición que hace posible la repetición o no de las acciones, sin la definición de una expresión inicial y una expresión final.

```
while (condicion) {  
    /*  
        Acciones a realizar cuando se cumple la condicion.  
        Accion ciclica.  
    */  
}
```

- **do while**

Esta estructura permite, a diferencia de los otros bucles, ejecutar las acciones una vez y luego evaluar la condición para ver si debe o no repetir dicha acción, iniciando así el bucle.

```
do {  
    /*  
    Acciones a realizar .  
    Se ejecutan por primera vez sin ver la condicion, luego para que repita  
    y se vuelva un bucle se fija que la condicion se cumpla  
    */  
} while (condicion);
```

3.7 Funciones Nativas

JavaScript nos provee de algunas funciones ya definidas que nos facilitan tener que crear algunas acciones más comunes, ayudándonos a reducir código y enfocarnos en resolver los programas que necesitamos crear.

Al estar ya definidas por JavaScript, solamente debemos invocar las funciones. A continuación, definiremos alguna de ellas a modo de repaso:

- **length()**: Función que nos permite ver la cantidad de valores que tiene un array.
- **console.log()**: Nos permite ver por consola los parámetros que le indiquemos.
- **alert()**: Nos permite ver en pantalla un mensaje de alerta.
- **parseInt()**: Transforma a número entero el parámetro ingresado

- **forEach()**: Función que nos permite recorrer un array y ejecutar lo que necesitemos a cada uno de los componentes del mismo.
 - a. **filter()**: Esta función nos ayuda a filtrar los elementos del array que cumplan con la condición, devolviendo un nuevo array.
 - b. **find()**: Esta función nos ayuda a buscar un elemento en particular. Tener en cuenta que va a devolver el primer elemento que cuente con esas características.
 - c. **map()**: Esta función nos permite aplicar una acción sobre cada elemento del array y obtener un nuevo array con los resultados sin eliminar el array original.

Conclusión

HTML, CSS y JavaScript son los tres lenguajes comprendidos por los navegadores, que nos servirán para armar un sitio web. HTML brinda la estructura, mientras que CSS nos ofrece la posibilidad de estilar el sitio y JavaScript de darle comportamiento.

Como repasamos en esta unidad , contamos con distintas herramientas, propiedades y métodos que nos permitirán darle vida a nuestros sitios haciéndolos estéticos y funcionales.

Ahora que contamos con los conceptos básicos ya frescos en nuestra memoria podemos iniciar en las siguientes unidades a ver algunas herramientas más avanzadas de estos lenguajes , agregados en las últimas versiones de los mismos, que nos ofrecen algunas opciones más específicas para agregar a nuestros sitios.



Bibliografía utilizada y sugerida

Artículos de revista en formato electrónico:

- **mdn web doc.** @keyframes. Disponible en:
<https://developer.mozilla.org/es/docs/Web/CSS/@keyframes>
- **Sara Cope.** “transform”. Disponible en:
<https://css-tricks.com/almanac/properties/t/transform/>
- **CSS Animations.** W3 School Tutoriales “Animations”. Disponible en:
https://www.w3schools.com/css/css3_animations.asp
- **CSS Animations.** W3 School Tutoriales “Animations”. Disponible en:
<https://css-tricks.com/almanac/properties/a/animation/>
- **CSS Animation.** W3 School Tutoriales “Transform”. Disponible en:
https://www.w3schools.com/css/css3_2dtransforms.asp
- **mdn web doc.** transition. Disponible en:
<https://developer.mozilla.org/es/docs/Web/CSS/transition>
- **Andrea Lluch Cruz.** Colores y fondos en CSS, atributos y propiedades
Disponible en: <https://www.eniun.com/atributos-propiedades-colores-css/>
- **mdn web doc.** DOM. Disponible en:
<https://developer.mozilla.org/es/docs/Glossary/DOM>
- **Desarrolloweb6.com** . Qué es el DOM. Disponible en:
<https://desarrolloweb.com/articulos/que-es-el-dom.html>

- **mdn web doc.** “¿Qué es el DOM?”. Disponible en:
https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model/Introduction